openbinary

Build an adder from logic gates and see how computers count, in four sessions.

PAN IIT OSS explainer • secondary school to diploma / undergraduate (levels 3-5)

Why this exists

Every computer stores numbers as rows of on/off switches and does arithmetic with a few kinds of logic gates. This explainer shows how counting in binary works and how an adder is built from gates, so you can build it yourself.

The end goal

Build an adder from logic gates and see how computers count. The end goal is a working adder built from logic gates, the heart of every processor: you see how a computer counts, how it stores negative numbers, why overflow happens, and why wide adders need clever carry circuits.

How the four sessions build up

1. **Session 1: Switches, lamps and counting in binary** (Level 1, conceptual interactive model, 60 min).
2. **Session 2: Number systems, logic gates and Boolean algebra** (Level 2, mathematical engineering model, 75 min).
3. **Session 3: Build an adder from gates and see how computers count** (Level 3, simulation and validation, 90 min).
4. **Session 4: A carry-lookahead adder** (Level 3, simulation and validation, 90 min).

1 Session 1: Switches, lamps and counting in binary

A. Real-world problem

A light switch is on or off, nothing in between. A computer is built from an enormous number of such switches, yet it stores photos, songs and bank balances. How can something that only knows 'on' and 'off' hold a number like 13? This session answers that with four lamps and four switches. It is a simplified educational model of how whole numbers are stored; real computers use the same idea with many more bits.

B. Concept and theory

In the decimal number 345 each digit is worth ten times the digit to its right: 3 hundreds, 4 tens, 5 ones. Binary does the same with the number 2. A *bit* is one digit that is 0 (switch off) or 1 (switch on). With four bits the places are worth 8, 4, 2 and 1 from left to right. The number is the sum of the place values of the switches that are on. So 1101 means $8 + 4 + 0 + 1 = 13$. Every switch you add doubles the number of patterns, because each old pattern can now come with the new switch off or on. Four switches make 16 patterns, which count from 0 (all off) to 15 (all on). Adding one to 15 would need a fifth switch; with only four, the count wraps round to 0.

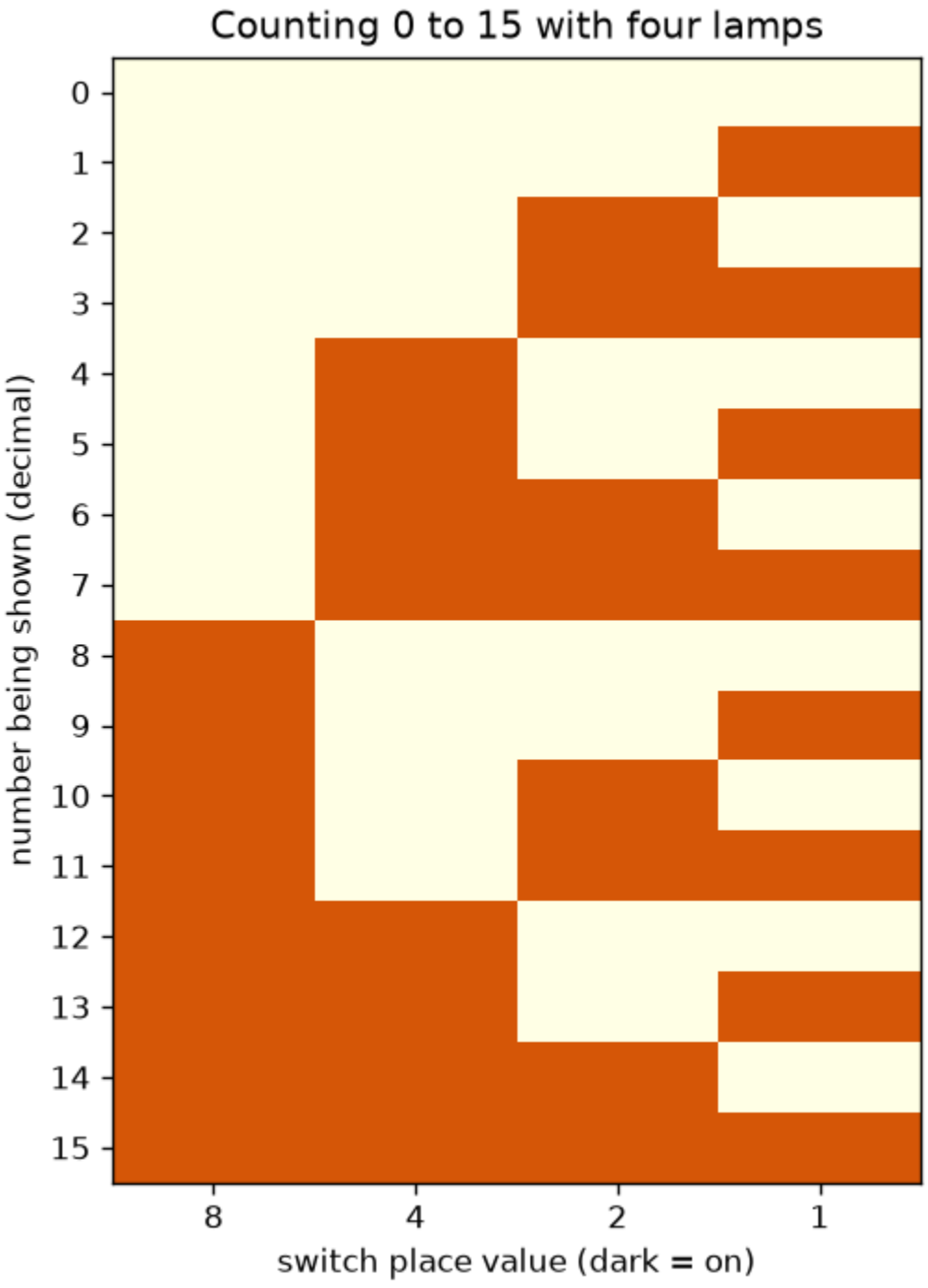
- b_k: state of switch number k: 0 (off) or 1 (on) (no unit (a bit))

- 2^k: place value of switch k (no unit)
- n: number of switches (bits) (count)
- N: the whole number the switches show (no unit)

$$N = \sum_{k=0}^{n-1} b_k \, 2^k$$

$$\text{number of patterns} = 2^n, \qquad \text{largest number} = 2^n - 1$$

$$1101_2 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13$$



Counting 0 to 15: one row per number, dark squares are lamps that are on.

E. Worked example

What number do the switches 1101 show? Then show the number 12 as switches.

1. Write the place values over the switches: 8, 4, 2, 1.
2. Switches on are the first, second and fourth, so add 8 + 4 + 1 and add 0 for the third: 8 + 4 + 0 + 1 = 13.
3. For 12 take the biggest place value that fits: 8 fits (12 - 8 = 4), 4 fits (4 - 4 = 0), 2 and 1 do not.
4. So 12 is 8 + 4 = 1100 (first and second switch on).
5. Check: 8 + 4 + 0 + 0 = 12.

Answer. 1101 shows 13, and 12 is written 1100.

F. Questions

1. **Conceptual.** Why does adding one more switch exactly double the number of different patterns?
2. **Numerical.** Read the switches 1011 as a decimal number, and write 12 as switches. Sanity check each result.
3. **Graph and simulation interpretation.** In the graph of lamps on against the number, how many numbers between 0 and 15 have exactly two lamps on? What is the highest point of the graph and where does it occur?
4. **Design and optimisation challenge.** Design: a hall has 100 seats numbered from 0 to 99. A sign should show the seat number on lamps. What is the smallest number of lamps? Name one trade-off of using more lamps than that.

2 Session 2: Number systems, logic gates and Boolean algebra

A. Real-world problem

A stairwell light has a switch at the top and one at the bottom, and either one must toggle the light. A lift must not move unless the door is closed AND the load is within the limit. Rules like these are built from logic gates, tiny circuits whose output is 0 or 1 and depends only on their inputs. To talk about the numbers inside such machines, engineers also write them in hexadecimal, which is binary in a short form. This is a simplified educational model: ideal gates with no electrical detail.

B. Concept and theory

In base b the digit string d_k ... d_1 d_0 means d_k b^k + ... + d_1 b + d_0. To convert a number to base b, divide by b again and again and collect the remainders, last remainder first. Hexadecimal (base 16) uses digits 0 to 9 and A to F (A is 10, F is 15), and each hex digit stands for exactly four bits, so 1011 0111 is B7. A logic gate maps input bits to an output bit. AND gives 1 only if both inputs are 1; OR gives 1 if at least one is 1; NOT flips a bit; XOR gives 1 if the inputs differ; NAND is NOT of AND. A truth table lists the output for every input row, and a claim that is true on every row is proved. De Morgan's laws turn an AND into an OR by flipping inputs and output. Because NAND(a, a) is NOT a, NAND can build NOT, then AND, then OR, then XOR: it is called universal. Any truth table can be turned into a circuit: write one AND term (a minterm) for each row whose output is 1 and OR the terms together (sum of products).

- b: the base of a number system (2, 10 or 16) (no unit)
- d_k: digit in place k (no unit)
- a, b: input bits of a gate (0 or 1)
- NOT a: the opposite bit of a (written with a bar in equations) (0 or 1)
- minterm: an AND term that is 1 on exactly one input row (0 or 1)

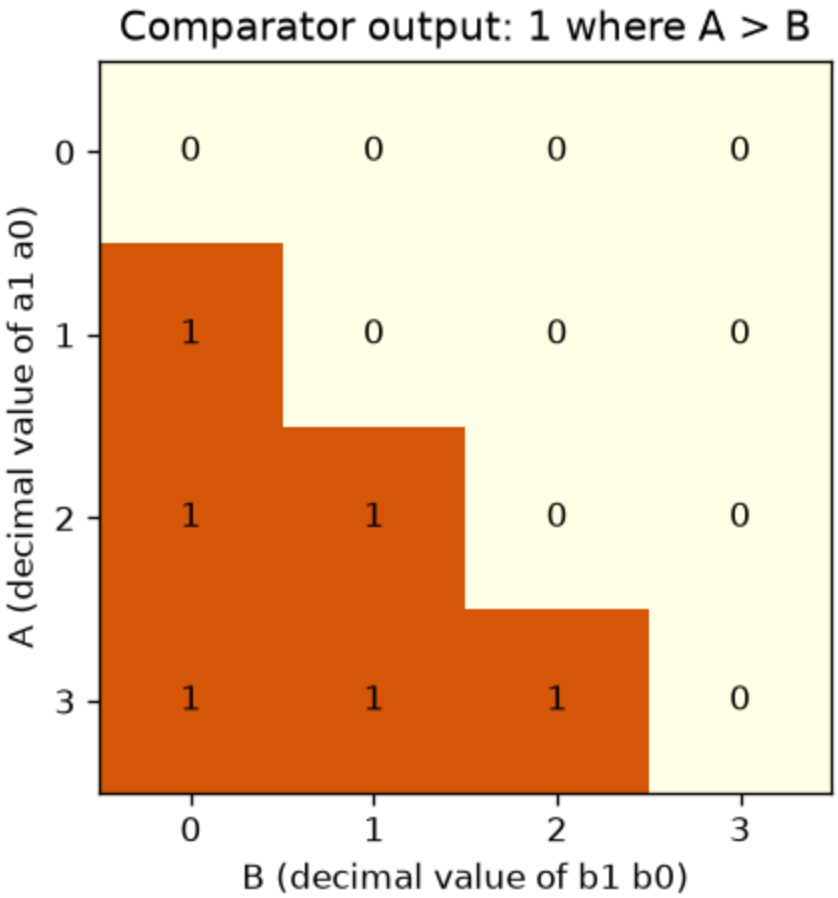
$$N = \sum_k d_k \, b^k$$

$$\overline{a \cdot b} = \bar{a} + \bar{b}, \qquad \overline{a + b} = \bar{a} \cdot \bar{b}$$

$$\bar{a} = \overline{a \cdot a}, \qquad a \cdot b = \overline{\overline{a \cdot b}}, \qquad a + b = \overline{\bar{a} \cdot \bar{b}}$$

$$a \oplus b = a\bar{b} + \bar{a}b$$

$$\text{majority}(a, b, c) = \bar{a}bc + a\bar{b}c + ab\bar{c} + abc = ab + ac + bc$$



The 2-bit comparator: dark where A is greater than B.

E. Worked example

Convert 183 to binary and to hexadecimal, and check by converting back.

1. Divide by 2 repeatedly and collect remainders from last to first. 183 = 128 + 32 + 16 + 4 + 2 + 1, so the bits are 1 0 1 1 0 1 1 1.
2. So 183 is 10110111 in binary.
3. Group in fours from the right: 1011 and 0111. 1011 is 11, which is B, and 0111 is 7.
4. So 183 is B7 in hexadecimal.
5. Check: B7 = 11 x 16 + 7 = 176 + 7 = 183. The same number.

Answer. 183 is 10110111 in binary and B7 in hexadecimal.

F. Questions

1. **Conceptual.** Why can NAND build every other gate? Show how to get NOT and OR from NAND.
2. **Numerical.** Convert the hexadecimal number A5 to decimal and to binary, and convert decimal 100 to binary and hexadecimal. Sanity check.
3. **Graph and simulation interpretation.** In the comparator figure (A down the side, B along the top, dark where A > B), where are the dark squares and how many are there? Why is the diagonal white?
4. **Design and optimisation challenge.** Design: a lock opens when the 2-bit number A equals the 2-bit number B. Build the truth table, write the sum-of-products form and give a shorter form with XNOR. How many minterms are there?

3 Session 3: Build an adder from gates and see how computers count

A. Real-world problem

Inside a processor there is no plus button, only gates. Yet it adds very many numbers every second, subtracts, and knows when an answer does not fit. How can a handful of AND, OR and XOR gates add two numbers, and why does it take longer to add 32 bits than 4? We build the arithmetic of a computer from scratch in a tiny simulator and test every circuit against Python's own integers on every possible input. This is a simplified educational model: ideal gates, each with one unit of delay, no clock and no real chip timing.

B. Concept and theory

A half adder adds two bits: the sum bit is a XOR b (1 when the inputs differ) and the carry bit is a AND b (1 when both are 1). A full adder also takes a carry in: sum = a XOR b XOR c_in and carry out = ab + (a XOR b) c_in; it is two half adders and an OR gate, five gates in all. Chain n full adders with the carry out of each feeding the next and you get an n-bit ripple-carry adder. In n-bit two's complement the leftmost bit counts as -2^(n-1) instead of +2^(n-1), so four bits cover -8 to 7 and 1111 means -1. To negate a number flip every bit and add 1, so a - b = a + (NOT b) + 1: XOR every b bit with a control bit `sub` and use `sub` as the carry in, and one adder both adds and subtracts. If the true answer is outside the range, the result wraps round and overflow occurs; it is detected when the carry into the top bit differs from the carry out of it. Each gate takes one unit of delay, and a wire's depth is one more than the depth of its slowest input. The carry out of stage i must wait for the carry out of stage i - 1, so the delay of the last carry grows in a straight line with the width. That is why fast processors use carry-lookahead circuits (a stretch topic) that work out carries in parallel. An ALU (arithmetic logic unit) feeds the same inputs to several operations and a control code selects which result comes out.

- a, b: input bits (or n-bit numbers) (0 or 1)
- c_in, c_out: carry into and out of a stage (0 or 1)
- n: adder width (bits)
- sub: control bit: 0 add, 1 subtract (0 or 1)
- T(n): delay of the last carry out (gate delays)
- op: ALU control code: 00 add, 01 subtract, 10 AND, 11 OR (2 bits)

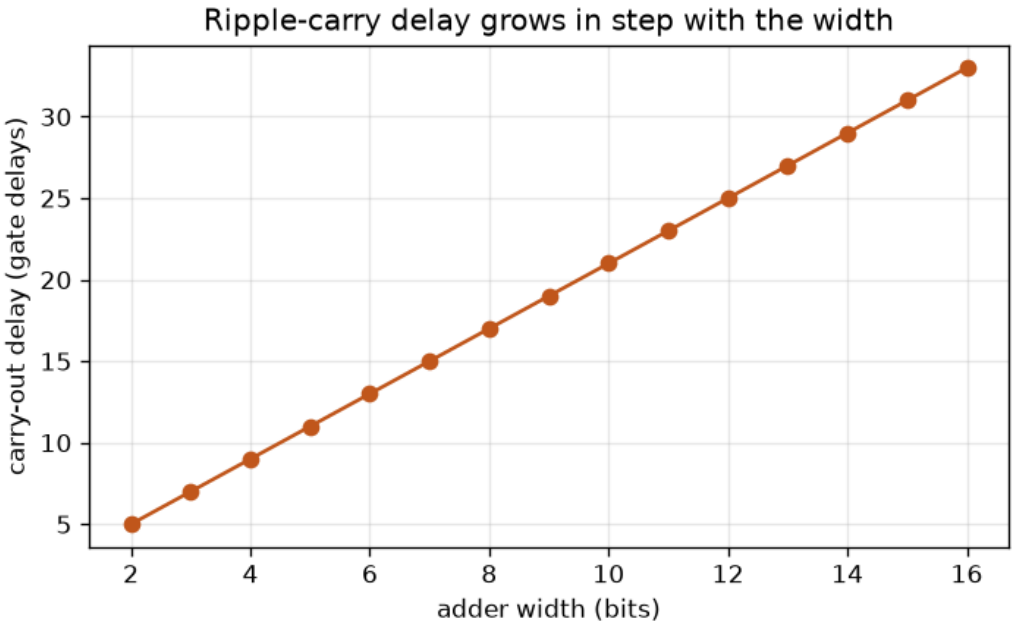
$$s = a \oplus b \oplus c_{in}, \qquad c_{out} = ab + (a \oplus b)\, c_{in}$$

$$\text{value} = -b_{n-1}2^{n-1} + \sum_{k=0}^{n-2} b_k 2^k, \qquad -2^{n-1} \le \text{value} \le 2^{n-1} - 1$$

$$a - b = a + \bar{b} + 1 \pmod{2^n}$$

$$\text{overflow} = c_{n-1} \oplus c_n$$

$$T(n) = 2n + 1 \text{ gate delays}$$



Carry-out delay of the ripple adder grows in a straight line with its width.

E. Worked example

Compute 6 - 3 on four bits with the adder-subtractor, showing the bits.

1. Subtraction means sub = 1: flip every bit of b and add 1 through the carry in.
2. 6 is 0110 and 3 is 0011; flipping the bits of 3 gives 1100.
3. Add 0110 + 1100 + 1 (the carry in) = 10011.
4. Keep only four bits: 0011, the carry out of the top bit is thrown away.
5. 0011 is 3, and 6 - 3 = 3. The carry into the top bit and the carry out of it are equal, so there is no overflow.

Answer. 6 - 3 = 3, pattern 0011, overflow flag 0.

F. Questions

1. **Conceptual.** Why does 7 + 1 give -8 on four bits, and how does the circuit know that the answer is wrong?
2. **Numerical.** Compute 5 - 7 on four bits: show the three numbers you add, the four-bit result, and whether overflow occurs. Sanity check with ordinary arithmetic.
3. **Graph and simulation interpretation.** From the delay graph, how many gate delays does the carry take for 8 bits, and for 12 bits? What does the slope of the line tell you?
4. **Design and optimisation challenge.** Design: each gate takes 0.5 ns and the carry out of a ripple adder must be ready within 20 ns. What is the widest adder allowed? What would you change to go wider?

4 Session 4: A carry-lookahead adder

A. Real-world problem

Session 3 built an adder that is correct for every input, but its carry ripples: the last carry waits for every carry before it, so the delay is 2n + 1 gate delays. A 16-bit ripple adder needs 33 gate delays and a 32-bit one needs 65, far too slow for the adder inside a processor that has to finish in one clock tick. How can an adder get its carries without waiting for them? We work out what each stage will generate and propagate, write every carry in terms of those signals, and build two faster adders in the same simulator. Each is checked against Python's own addition, and we count honestly what the speed costs in gates. This is a simplified educational model: ideal gates with two inputs and one unit of delay each, no wires, no fan-out limit, no clock and no real chip timing.

B. Concept and theory

Look at one stage with inputs a and b and ask what it does to a carry that arrives. If a = b = 1 it **generates** a carry whatever arrives: g = a AND b. If a and b differ it **propagates**: an arriving carry passes straight through, so p = a XOR b. If a = b = 0 it **kills** an arriving carry, and g = p = 0. The carry out of the stage is c_next = g OR (p AND c), and the sum bit is p XOR c. The key fact is that g and p use only the stage's own inputs, so they are ready after one gate delay in every stage at once.

Now substitute c_next = g + p c into itself. For example c4 = g3 + p3 g2 + p3 p2 g1 + p3 p2 p1 g0 + p3 p2 p1 p0 c_in (+ is OR, writing two symbols together is AND). Every term uses only g, p and the carry in, so all carries can be computed at the same time instead of one after another. At most one term can be 1, because a stage that generates has p = 0 and blocks every term that starts further down. So each carry has exactly one source: a stage that generated it with every stage above it passing it on, or the carry in itself.

With two-input gates an AND of 17 signals still needs 5 levels, and an OR of 17 terms another 5, so lookahead is not instant. The **flat lookahead adder** builds every term and every OR in full, joining the shallowest wires first; it is the textbook idea with no sharing, fast but expensive in gates. The **parallel-prefix adder** is smarter. A group of stages has its own generate G and propagate P, and a high group joins the group below it with G = G_high + P_high G_low and P = P_high P_low. Joining at distances 1, 2, 4, 8, ... (the Kogge-Stone pattern) makes every stage's group reach back to stage 0 after log2(n) rounds, two gate delays per round, so doubling the width adds exactly 2 gate delays. No circuit of two-input gates can do better than ceil(log2(2n

+ 1)) levels for the last carry, because that carry depends on all 2n + 1 inputs.

- a, b: input bits of one stage (0 or 1)
- g, p: generate (a AND b) and propagate (a XOR b) of a stage (0 or 1)
- c_in, c_i: carry in, and carry into stage i (0 or 1)
- G, P: generate and propagate of a group of stages (0 or 1)
- n: adder width (bits)
- T(n): delay of the last carry out (gate delays)
- gates: number of gates in the circuit (count)

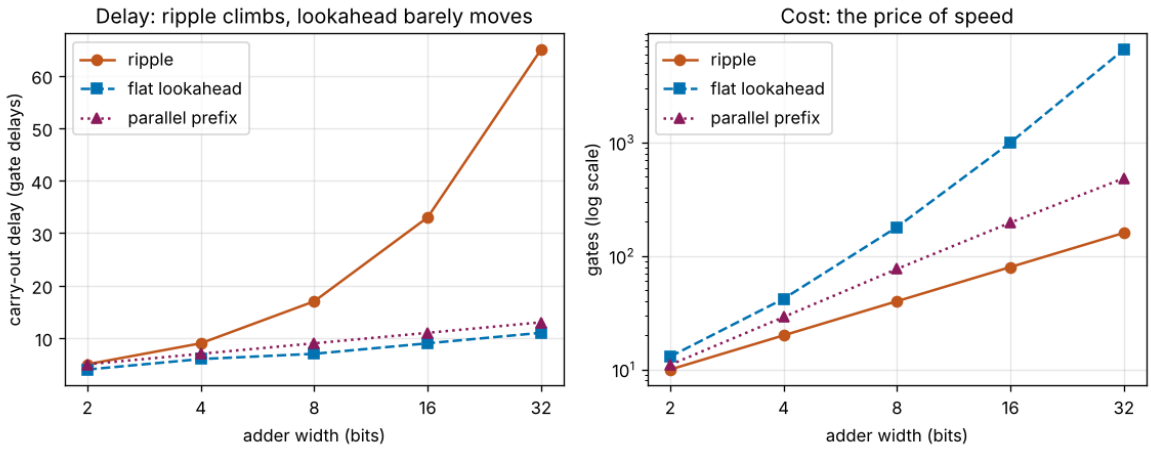
$$g_i = a_i b_i, \qquad p_i = a_i \oplus b_i$$

$$c_{i+1} = g_i + p_i c_i, \qquad s_i = p_i \oplus c_i$$

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i \cdots p_0 \, c_{in}$$

$$(G, P) = (G_h + P_h G_l,\ P_h P_l)$$

$$T_{\text{ripple}}(n) = 2n + 1, \qquad T_{\text{prefix}}(2^k) = 2k + 3, \qquad T \geq \lceil \log_2(2n + 1) \rceil$$



Carry-out delay and gate count against width: the lookahead adders are much faster and cost more gates.

E. Worked example

Add 11 + 5 on four bits using the lookahead equations, with carry in 0, and say where the last carry comes from.

1. Write the inputs: a = 1011 and b = 0101.
2. Generate g = a AND b = 0001 (only stage 0 has two ones) and propagate p = a XOR b = 1110 (stages 1, 2 and 3 have one 1 each).
3. All the carries from the equations: c1 = g0 + p0 c_in = 1 + 0 = 1; c2 = g1 + p1 g0 + p1 p0 c_in = 0 + 1 + 0 = 1; c3 = g2 + p2 g1 + p2 p1 g0 + ... = 0 + 0 + 1 = 1; c4 = g3 + p3 g2 + p3 p2 g1 + p3 p2 p1 g0 + ... = 0 + 0 + 0 + 1 = 1.
4. The sum bits are s_i = p_i XOR c_i, with c_0 = c_in = 0: s0 = 0 XOR 0 = 0, s1 = 1 XOR 1 = 0, s2 = 1 XOR 1 = 0, s3 = 1 XOR 1 = 0.

5. The result is 0000 with carry out 1, which is 16, and 11 + 5 = 16. The only term of c4 that is 1 is p3 p2 p1 g0: stage 0 generated the carry and stages 1, 2 and 3 passed it on.

Answer. 11 + 5 = 16: sum bits 0000, carry out 1, and the last carry comes from stage 0.

F. Questions

1. **Conceptual.** Why can a lookahead adder find the carry out of stage 3 without waiting for the carries out of stages 0, 1 and 2? What do generate and propagate mean?
2. **Numerical.** Use the equations to find the carry out of stage 3 (c4) for a = 1101 (13) and b = 0111 (7) with carry in 0. Which stage did it come from? Sanity check with ordinary addition.
3. **Graph and simulation interpretation.** From the delay graph, how many gate delays does the carry out take at 16 bits and at 32 bits for the ripple, flat and prefix adders? What happens to each when the width doubles?
4. **Design and optimisation challenge.** Design: a 32-bit adder must have its carry out ready within 7.5 ns at 0.5 ns per gate delay, using at most 1000 gates. Which of the ripple, flat and prefix designs would you choose? What would change if the budget were 10 gate delays?

5 Real-world mini-project

Build and test an 8-bit adder-subtractor. A small calculator chip needs to add and subtract 8-bit numbers (-128 to 127) and to warn when the answer does not fit. Use `build_add_sub(8)` and `add_sub` from Session 3. Test it on every pair of inputs for both operations against Python integers, count the pairs that overflow, count its gates, read the delay of its overflow flag, and then decide the widest adder-subtractor whose overflow flag is ready within a 10 ns budget if one gate takes 0.5 ns.

Choose the adder for a 32-bit processor. A small processor needs a 32-bit adder whose carry out must be ready within 7.5 ns, at 0.5 ns per gate delay (15 gate delays), and the chip area allows at most 1000 gates for the adder. Use `build_ripple`, `build_flat` and `build_prefix` from Session 4 at 32 bits. Check each design against Python, read its gates and carry-out delay, convert the delay to nanoseconds, and decide which design meets both budgets. **Deliverable:** A script that builds the three 32-bit adders, checks each against Python on 20000 random pairs plus corner cases and the prefix adder on 200000, prints a table of gates, carry-out delay and time in nanoseconds, prints which designs meet 15 gate delays and 1000 gates, finds the widest ripple adder within 15 gate delays, and saves a picture of the delay and gate curves. **Success looks like:** No mismatches in any check. The table shows ripple 160 gates, 65 gate delays, 32.5 ns; flat 6608 gates, 11 gate delays, 5.5 ns; prefix 485 gates, 13 gate delays, 6.5 ns. Only the prefix adder meets both budgets (ripple is too slow, flat uses too many gates), the widest ripple adder within 15 gate delays is 7 bits, and the prefix adder is 5 times faster than ripple (65 / 13) for about 3 times the gates (485 / 160).

6 The path ahead

1. **Session 5: A 4-bit multiplier.** A 4 x 4 multiplier made of gates with an 8-bit product.
2. **Session 6: A saturating adder.** An 8-bit adder-subtractor that returns the nearest extreme on overflow.
3. **Session 7: A counter that remembers.** A 4-bit counter driven by clock ticks.
4. **Session 8: An ALU with flags and a decoder.** An 8-bit ALU with four operations and flags, and a decoder lighting the right segments for each digit.

7 Answers and hints

Session 1

1. **Conceptual.** Hint: Take every pattern you already had and ask how many ways the new switch can be set. Solution: Each old pattern can be combined with the new switch off or on, which gives two new patterns for every old one. Four switches make 16 patterns, five make 32. So n switches make 2^n patterns, and the largest number is 2^n - 1 because the counting starts at 0.
2. **Numerical.** Hint: Use the place values 8, 4, 2, 1. Solution: 1011 = 8 + 0 + 2 + 1 = 11. For 12: 8 + 4 = 12, so the pattern is 1100. Sanity check: 11 is below 15 (the largest on 4 switches) and 1100 sums back to 12.
3. **Graph and simulation interpretation.** Hint: Count the dots at height 2, and look for the number with every switch on. Solution: Six numbers have exactly two lamps on: 3, 5, 6, 9, 10 and 12. The graph is highest (4 lamps) at 15, where every switch is on, and lowest (0 lamps) at 0. The counts at heights 0 to 4 are 1, 4, 6, 4, 1.
4. **Design and optimisation challenge.** Hint: Find the smallest n with 2^n at least 100. Solution: 6 lamps give only 64 patterns, which is not enough. 7 lamps give 128 patterns, which covers 0 to 99, so 7 lamps are enough and 7 is the smallest number. Using 8 or more lamps would cost more parts and more wiring while only wasting patterns; using fewer is impossible. Assumption: one lamp per bit and a fixed, known maximum seat number.

Session 2

1. **Conceptual.** Hint: What does NAND do when both inputs are the same wire? Then use De Morgan. Solution: NAND(a, a) is NOT(a AND a) = NOT a, so NAND gives NOT. AND is NOT of NAND, so AND takes two NAND gates. OR(a, b) = NOT(NOT a AND NOT b) = NAND(NOT a, NOT b), which takes three NAND gates. XOR takes four. With NOT and AND you can build any truth table, so NAND alone is enough.
2. **Numerical.** Hint: A is 10. Each hex digit is four bits. Solution: A5 = 10 x 16 + 5 = 165, and A = 1010, 5 = 0101, so A5 is 10100101. For 100: 64 + 32 + 4 = 100, so it is 1100100 in binary; grouping 0110 0100 gives 64 in hexadecimal (6 x 16 + 4 = 100). Check: 10100101 = 128 + 32 + 4 + 1 = 165.
3. **Graph and simulation interpretation.** Hint: On the diagonal A and B are equal. Solution: There are six dark squares, all below the diagonal (the row number A is larger than the column number B): 3 + 2 + 1 = 6. The diagonal is white because there A = B, so A > B is false. The squares above the diagonal are white because there B is larger.
4. **Design and optimisation challenge.** Hint: List the 16 rows (a1 a0 b1 b0) and mark those with A = B. XNOR is 1 when its inputs are the same. Solution: A = B on four rows: 0000, 0101, 1010, 1111, so there are 4 minterms out of 16. Sum of products: the OR of those four 4-input AND terms. Shorter: (a1 XNOR b1) AND (a0 XNOR b0), because A = B exactly when each bit pair matches. Trade-off: the sum of products is mechanical and always works, the short form uses fewer gates but needs insight. We checked both on all 16 rows.

Session 3

1. **Conceptual.** Hint: Write 0111 + 0001, and compare the carry into the top bit with the carry out of it. Solution: 0111 + 0001 = 1000. Read as two's complement, 1000 is -8, because the top bit has the weight -8. The true answer 8 does not fit in -8 to 7, so the result wrapped round. The carry into the top bit is 1 but the carry out of it is 0; they differ, so the overflow flag is 1.
2. **Numerical.** Hint: Flip the bits of 7 (0111), add 0101 and add the carry in 1. Solution:

NOT 0111 = 1000. Add 0101 + 1000 + 1 = 1110 (14). Read as two's complement, 1110 is 14 - 16 = -2. Check: 5 - 7 = -2, and -2 is inside -8 to 7, so overflow is 0.

3. **Graph and simulation interpretation.** Hint: The line is T(n) = 2n + 1. Solution: For 8 bits T = 17 gate delays; for 12 bits T = 2 x 12 + 1 = 25. The slope is 2 gate delays per extra bit, the two gates (AND then OR) that the carry passes through in every stage. Doubling the width roughly doubles the delay, which is the reason for carry-lookahead.
4. **Design and optimisation challenge.** Hint: Solve (2n + 1) x 0.5 <= 20. Solution: (2n + 1) x 0.5 <= 20 gives 2n + 1 <= 40, so n <= 19.5 and the widest is 19 bits (39 gate delays = 19.5 ns; 20 bits would need 41 gate delays = 20.5 ns). To go wider, use a carry-lookahead adder, or split the work over two clock cycles. Trade-off: lookahead needs more gates for less delay.

Session 4

1. **Conceptual.** Hint: Write c4 with the substitution c_next = g + p c, and see which signals it uses. Solution: A stage generates a carry when a = b = 1 (g = a AND b) and propagates one when a and b differ (p = a XOR b); both depend only on that stage's own inputs, so every stage has them after one gate delay. Substituting c_next = g + p c again and again gives c4 = g3 + p3 g2 + p3 p2 g1 + p3 p2 p1 g0 + p3 p2 p1 p0 c_in. This uses only g, p and the carry in, never c1, c2 or c3, so it can be built next to them instead of after them. The price is more gates.
2. **Numerical.** Hint: First find g = a AND b and p = a XOR b, bit by bit. Solution: a = 1101 and b = 0111, so g = 0101 (stages 0 and 2 have two ones) and p = 1010 (stages 1 and 3 have one 1). Then c4 = g3 + p3 g2 + p3 p2 g1 + p3 p2 p1 g0 + p3 p2 p1 p0 c_in = 0 + 1 + 0 + 0 + 0 = 1, and the term that is 1 is p3 g2: stage 2 generated a carry and stage 3 passed it on. Check: 13 + 7 = 20 = 16 + 4, so there is a carry out of the top stage and the four sum bits are 0100.
3. **Graph and simulation interpretation.** Hint: Read the three lines at 16 and at 32 and subtract. Solution: At 16 bits: ripple 33, flat 9, prefix 11. At 32 bits: ripple 65, flat 11, prefix 13. Doubling the width from 16 to 32 adds 32 gate delays to the ripple adder (it nearly doubles, since T = 2n + 1) but only 2 to each lookahead adder, because one more round of combining is enough. So the gap between ripple and lookahead keeps widening with the width.
4. **Design and optimisation challenge.** Hint: Convert the time to gate delays first: 7.5 / 0.5. Solution: 7.5 / 0.5 = 15 gate delays. Ripple takes 65 (160 gates), so it is too slow. Flat takes 11 gate delays but 6608 gates, so it is too big. Prefix takes 13 gate delays with 485 gates, so only the prefix adder meets both limits. With a budget of 10 gate delays none of the three designs works at 32 bits (the fastest, flat, needs 11), so you would have to narrow the adder (flat at 16 bits needs 9) or find a different design; even then no circuit of two-input gates can beat ceil(log2 65) = 7 levels for a 32-bit carry out.

8 How to join

Read `CONTRIBUTING.md` in the repository, then pick an issue labelled `good first issue`. Repository: https://github.com/KrunalPanchal1995/openbinary